Thanaphon Naksri 6634421523

Intro to Computational Linguistics: PA1

Question 1: precision, recall, F1-score for each label + overall accuracy

Using the "classification_report" function from scikit-learn, these are the evaluations:

```
Logistic Regression Performance:
                            precision    recall  f1-score   support

            Analyst Update       0.98      0.63      0.77        73
     Company | Product News       0.80      0.88      0.84       852
                Currencies       0.88      0.66      0.75        32
                  Dividend       0.98      0.97      0.97        97
                  Earnings       0.94      0.92      0.93       242
               Energy | Oil       0.82      0.77      0.79       146
        Fed | Central Banks       0.85      0.82      0.84       214
                Financials       0.86      0.85      0.85       160
     General News | Opinion       0.73      0.74      0.74       336
  Gold | Metals | Materials       0.38      0.46      0.41        13
                        IPO       0.83      0.71      0.77        14
        Legal | Regulation       0.93      0.72      0.82       119
         M&A | Investments       0.83      0.59      0.69       116
                     Macro       0.78      0.83      0.80       415
                   Markets       0.84      0.74      0.78       125
          Personnel Change       0.93      0.77      0.84       112
                  Politics       0.93      0.86      0.90       249
          Stock Commentary       0.78      0.90      0.83       528
            Stock Movement       0.82      0.72      0.77       197
Treasuries | Corporate Debt       0.95      0.73      0.82        77

                   accuracy                           0.82      4117
                  macro avg       0.84      0.76      0.80      4117
               weighted avg       0.83      0.82      0.82      4117
```

<u>The next questions (Q2 & Q3) are identified using these codes</u>

```python
report_dict = classification_report(test_df['label'], predictions_lr, output_dict=True)
report_df = pd.DataFrame(report_dict).transpose()
# Settings
n=2
metric ="f1-score"
# Remove summary rows to analyze only specific labels
label_perf = report_df.drop(['accuracy', 'macro avg', 'weighted avg'])
topNlabel = label_perf.sort_values(by=metric, ascending=False).head(n)
botNlabel = label_perf.sort_values(by=metric, ascending=True).head(n)

# Q2: 2 best performing labels (dividend & earning reports literally almost always include words such as 'dividend' and 'earning')
print(f"Top {n} performing labels (by {metric}):")
display(topNlabel)
print_specific_top_features(pipeline=pipe, class_labels=list(topNlabel.index), n=10)
print ("----"*30)

# Q3: 2 worst performming labels -> low training data, might look into captured characteristics (like distinct or overused words fo
print("\nBottom 2 performing labels (by f1-score):")
display(botNlabel)
print_specific_top_features(pipeline=pipe, class_labels=list(botNlabel.index), n=10)
```

Main code for making a report on these questions

```python
def print_specific_top_features(pipeline, class_labels, n=10):
    vectorizer = pipeline.named_steps['vectorizer']
    classifier = pipeline.named_steps['lr_clf']
    feature_names = vectorizer.get_feature_names_out()

    classes = classifier.classes_
    coefs = classifier.coef_

    # If a single string is passed, wrap it in a list
    if isinstance(class_labels, str):
        class_labels = [class_labels]

    print("\n--- Feature Importance by Class ---")
    for class_label in class_labels:
        if class_label in classes:

            i = list(classes).index(class_label) # Find the index of the class label in the model's classes

            # Get indices of top n largest coefficients
            top_indices = coefs[i].argsort()[-n:][::-1]
            top_features = [feature_names[j] for j in top_indices]
            print(f"\nLabel: {class_label}")
            print(f"Top {n} words: {', '.join(top_features)}")
        else:
            print(f"\nLabel: {class_label} not found in model.")
```

Function for looking into the trained model's weight for a specific class

Question 2: Two best-performing labels & top 10 highest-weighted features

Top 2 performing labels (by f1-score):

	precision	recall	f1-score	support
Dividend	0.979167	0.969072	0.974093	97.0
Earnings	0.936709	0.917355	0.926931	242.0

```
--- Feature Importance by Class ---

Label: Dividend
Top 10 words: dividend, declares, distribution, announces, dividends, distributions, quarterly, trust, monthly, reit

Label: Earnings
Top 10 words: earnings, results, conference, q2, reports, 2022, quarter, upcoming, preview, profit
```

They include words that are specific to them, like dividend, earnings, results, quarterly terms etc.

Also, "Earnings" has a relatively high support, meaning they are trained on a lot

Question 3: Two worst-performing labels & top 10 highest-weighted features

Bottom 2 performing labels (by f1-score):

	precision	recall	f1-score	support
Gold | Metals | Materials	0.375000	0.461538	0.413793	13.0
M&A | Investments	0.831325	0.594828	0.693467	116.0

```
--- Feature Importance by Class ---

Label: Gold | Metals | Materials
Top 10 words: gold, metals, copper, settlement, dollar, comdx, prices, nickel, trading, investors

Label: M&A | Investments
Top 10 words: acquires, acquire, acquisition, merger, sale, stake, invests, buy, purchase, spin
```

Gold etc. has a significantly low amount of data so the model doesn't have enough information to learn

M&A | Investment included common word in finance also found in other classes such as sale, buy, purchase

Question 4: Prompt I've ended up using (for both predicting news by news & batch)

First, Define the classes we want the prediction to be (using unique() on the label column)

Also, make a detailed description of each class for the LLM to "understand"

```
labels = train_df['label'].unique().tolist()
label_desc_map = {
    'Company | Product News': 'Announcements about new products, services, store openings, or general company updates.',
    'Stock Commentary': 'General market commentary, opinion pieces on stock performance, or analyst sentiment.',
    'Macro': 'Broad economic news, including inflation, GDP, unemployment, consumer spending, and recession fears.',
    'General News | Opinion': 'Non-financial general news, political opinion pieces, or broad market editorials.',
    'Earnings': 'Reports on quarterly or annual financial results, revenue, profits, EPS, and earnings calls.',
    'Politics': 'News involving government officials, elections, legislation, or geopolitical events affecting markets.',
    'Fed | Central Banks': 'News about the Federal Reserve, ECB, interest rate decisions, and central bank speeches.',
    'Stock Movement': 'Reports on specific stocks rising or falling, market movers, and daily stock performance.',
    'Financials': 'News specifically related to banks, insurance companies, fintech, and the financial sector.',
    'Energy | Oil': 'News about oil prices, energy companies, renewable energy, and gas markets.',
    'Markets': 'Broad stock market updates, index performance (S&P 500, Nasdaq), and market trends.',
    'Personnel Change': 'News about CEO appointments, resignations, board changes, or key executive hires.',
    'Legal | Regulation': 'Lawsuits, regulatory investigations, antitrust actions, or court rulings involving companies.',
    'M&A | Investments': 'Mergers, acquisitions, buyouts, stake purchases, and venture capital investments.',
    'Dividend': 'Announcements regarding dividend declarations, increases, cuts, or payout dates.',
    'Treasuries | Corporate Debt': 'News about government bonds, corporate bonds, yields, and debt issuance.',
    'Analyst Update': 'Specific analyst ratings (buy/sell/hold), price target changes, or upgrades/downgrades.',
    'Currencies': 'News about foreign exchange rates, the US Dollar, Euro, Yen, or crypto movements.',
    'Gold | Metals | Materials': 'News about commodities like gold, silver, copper, steel, and mining companies.',
    'IPO': 'News about Initial Public Offerings, companies going public, or listing announcements.'
}
```

Prompt Structure ("input_text_block" is the concatenation of input data according to batch size)

```
    prompt = f"""You are an expert financial news classifier.
Classify the following {len(texts)} news items into exactly one of these categories:

{label_list_str}

Input Items:
{input_text_block}

Task:
Return a numbered list of categories corresponding to the input news items.
Format:
1. Category Name
2. Category Name
...

Do NOT provide explanations. JUST the numbered list."""
```

Full Code (Including helper functions for batch processing)

```python
# TODO: batch
# Zero-shot LLM with Gemini (Batch Processing)
import time
import math
from tqdm.auto import tqdm
from google import genai
import os
from dotenv import load_dotenv

load_dotenv()

# 1. Setup Client
client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))

# 2. Labels & Descriptions
labels = train_df['label'].unique().tolist()
label_desc_map = {
    'Company | Product News': 'Announcements about new products, services, store openings, or general company updates.',
    'Stock Commentary': 'General market commentary, opinion pieces on stock performance, or analyst sentiment.',
    'Macro': 'Broad economic news, including inflation, GDP, unemployment, consumer spending, and recession fears.',
    'General News | Opinion': 'Non-financial general news, political opinion pieces, or broad market editorials.',
    'Earnings': 'Reports on quarterly or annual financial results, revenue, profits, EPS, and earnings calls.',
    'Politics': 'News involving government officials, elections, legislation, or geopolitical events affecting markets.',
    'Fed | Central Banks': 'News about the Federal Reserve, ECB, interest rate decisions, and central bank speeches.',
    'Stock Movement': 'Reports on specific stocks rising or falling, market movers, and daily stock performance.',
    'Financials': 'News specifically related to banks, insurance companies, fintech, and the financial sector.',
    'Energy | Oil': 'News about oil prices, energy companies, renewable energy, and gas markets.',
    'Markets': 'Broad stock market updates, index performance (S&P 500, Nasdaq), and market trends.',
    'Personnel Change': 'News about CEO appointments, resignations, board changes, or key executive hires.',
    'Legal | Regulation': 'Lawsuits, regulatory investigations, antitrust actions, or court rulings involving companies.',
    'M&A | Investments': 'Mergers, acquisitions, buyouts, stake purchases, and venture capital investments.',
    'Dividend': 'Announcements regarding dividend declarations, increases, cuts, or payout dates.',
    'Treasuries | Corporate Debt': 'News about government bonds, corporate bonds, yields, and debt issuance.',
    'Analyst Update': 'Specific analyst ratings (buy/sell/hold), price target changes, or upgrades/downgrades.',
    'Currencies': 'News about foreign exchange rates, the US Dollar, Euro, Yen, or crypto movements.',
    'Gold | Metals | Materials': 'News about commodities like gold, silver, copper, steel, and mining companies.',
    'IPO': 'News about initial Public Offerings, companies going public, or listing announcements.'
}

# 3. Create Batch Prompt
def get_batch_prompt(texts):
    # Create the label list
    label_list_str = "\n".join([f"- {lbl}: {desc}" for lbl, desc in label_desc_map.items()])

    # Create the numbered input list
    input_text_block = ""
    for idx, text in enumerate(texts):
        input_text_block += f"News {idx+1}: {text}\n"

    prompt = f"""You are an expert financial news classifier.
Classify the following {len(texts)} news items into exactly one of these categories:

{label_list_str}

Input Items:
{input_text_block}

Task:
Return a numbered list of categories corresponding to the input news items.
Format:
1. Category Name
2. Category Name
...

Do NOT provide explanations. JUST the numbered list."""
    return prompt

# 4. Predict Wrapper
def get_batch_predictions(texts):
    prompt = get_batch_prompt(texts)
    try:
        response = client.models.generate_content(
            model="gemini-2.0-flash-exp", # or gemini-1.5-flash
            contents=prompt
        )
        return response.text.strip()
    except Exception as e:
        print(f"Batch Error: {e}")
        return ""
```

```python
# 5. Parsing Logic
def parse_batch_response(response_text, batch_size):
    predictions = []
    lines = response_text.split('\n')

    for line in lines:
        stripped = line.strip()
        # Look for "1. label" or "1.label" format
        if stripped and stripped[0].isdigit():
            parts = stripped.split('.', 1)
            if len(parts) > 1:
                # Clean up the label part
                raw_label = parts[1].strip()
                predictions.append(raw_label)

    # Validation: If we missed some or got extra, pad or truncate
    if len(predictions) < batch_size:
        print(f"Warning: Expected {batch_size} preds, got {len(predictions)}. Padding with 'Error'.")
        predictions.extend(["Error"] * (batch_size - len(predictions)))

    return predictions[:batch_size]

# 6. Execution Loop
# Use full test_df for batching as it's faster
target_df = test_df.copy() # or test_df.sample(200) for quick test

batch_size = 20 # 20 items per call
all_predictions = []

print(f"Processing {len(target_df)} items in batches of {batch_size}...")

# tqdm range based loop
num_batches = math.ceil(len(target_df) / batch_size)
for i in tqdm(range(num_batches)):
    start_idx = i * batch_size
    # Get the chunk of text
    batch_texts = target_df['text'].iloc[start_idx : start_idx + batch_size].tolist()

    if not batch_texts: break

    # Call API
    raw_response = get_batch_predictions(batch_texts)

    # Parse
    parsed_preds = parse_batch_response(raw_response, len(batch_texts))
    all_predictions.extend(parsed_preds)

    # Rate limit Safety:
    # Gemini Free Tier is ~15 requests per minute.
    # 4 seconds sleep = 15 requests/min.
    time.sleep(4)

target_df['llm_pred'] = all_predictions

# 7. Clean and Save
def clean_label(pred):
    for label in labels:
        if label.lower() in pred.lower():
            return label
    return "Unknown" # fallback

target_df['llm_pred_clean'] = target_df['llm_pred'].apply(clean_label)

target_df.to_csv("pred_gemini_batch_full.csv")
print("Finished & Saved!")

# Show sample
display(target_df[['text', 'label', 'llm_pred_clean']].head())
```

Question 5: Zero-shot LLM better or worse?

These are the same classifications report for predictions made on test.csv using ***Typhoon***

(typhoon-v2.5-30b-a3b-instruct)

```
Zero-shot LLM Performance
There are 42 predictions with hallucinated label
                              precision    recall  f1-score   support

Treasuries | Corporate Debt       1.00      0.30      0.46        77
           Stock Commentary       0.63      0.80      0.70       528
                 Currencies       0.63      0.75      0.69        32
         Legal | Regulation       0.49      0.93      0.64       119
                   Dividend       0.88      0.96      0.92        97
                    Markets       0.52      0.67      0.59       125
                   Politics       0.64      0.94      0.76       249
                 Financials       0.00      0.00      0.00       160
     Company | Product News       0.90      0.60      0.72       852
                      Macro       0.87      0.24      0.38       415
        Fed | Central Banks       0.93      0.64      0.76       214
            Stock Movement       0.47      0.59      0.52       197
                   Earnings       0.56      0.93      0.70       242
               Energy | Oil       0.58      0.83      0.69       146
                        IPO       0.43      0.93      0.59        14
          M&A | Investments       0.44      0.89      0.59       116
  Gold | Metals | Materials       0.53      0.62      0.57        13
           Personnel Change       0.77      0.92      0.84       112
            Analyst Update       0.38      0.37      0.38        73
     General News | Opinion       0.48      0.39      0.43       336

                  micro avg       0.63      0.63      0.63      4117
                  macro avg       0.61      0.66      0.60      4117
               weighted avg       0.68      0.63      0.61      4117
```

As seen in the 2 reports, **Typhoon v2.5** performs approximately **25% worse** in weighted f1-score on test.csv

This might be due to the LLM not being specifically tuned for this domain & dataset.

Also, some labels might be ambiguous & confusing to LLM like:

- "Gold | Metals | Materials" vs "Commodities"
- "M&A | Investments" vs "Company | Product News"

In terms of computational resources used for inference, typhoon takes around 15- 30 minutes to make predictions on 4117 news found in test.csv while logistic regression only takes seconds to train the model & predict

For cost, LR is free to use while LLM cost money per token spent in most cases or require expensive gear to host powerful model locally